Guia: rodar e testar o projeto localmente do zero

Sep 30, 2026 · @Nickolas

Visão geral

Para rodar tudo localmente, você precisa de três coisas instaladas (Node, pnpm e Docker) e de uma única conta obrigatória: a API da Anthropic, com créditos. Todo o resto roda na sua máquina ou é opcional. Conte com 1 a 2 horas na primeira vez, a maior parte esperando downloads.

Este guia usa o **Supabase local** (dentro do Docker): banco, autenticação, arquivos e caixa de e-mails de teste ficam no seu computador, sem limite de envio e sem risco para dados reais. Os comandos estão no formato do **Git Bash no Windows**; no macOS e no Linux, são os mesmos no terminal.

Componente	O que faz	Como roda localmente	Obrigatório?
App web (Next.js)	Telas, login, upload, análise, painel	`pnpm dev`, em http://localhost:3000	Sim
Supabase	Banco Postgres, autenticação, arquivos, fila (pgmq)	`pnpm db:start`, no Docker	Sim
Worker	OCR, conversão de DOCX, análise com IA, avisos	`pnpm worker:start`, no Docker	Sim
Anthropic (Claude)	Extração, riscos, prazos e resumo	API na nuvem, com a sua chave	Sim
Mailpit	Recebe os e-mails de login do Supabase local	Vem com o Supabase local	Automático
Stripe (modo de teste)	Planos, checkout e portal de pagamento	Stripe CLI repassa o webhook	Só para testar planos
Brevo (SMTP)	Envio real de e-mails de aviso	Serviço na nuvem	Não
Sentry	Monitoramento de erros	Serviço na nuvem	Não

A ordem do guia é a ordem de execução: instalar ferramentas, criar contas, obter o código, subir o Supabase, preencher as variáveis de ambiente, subir o worker e o app, e só então as integrações opcionais e os testes.

Passo 1: instalar as ferramentas

Instale nesta ordem e confira cada uma com o comando indicado antes de seguir.

1. **Git e Git Bash** (Windows): baixe em https://git-scm.com/download/win e instale com as opções padrão. No macOS e no Linux, o Git já costuma vir instalado.
   - Conferir: `git --version`
2. **Node.js 22 (LTS)**: baixe em https://nodejs.org. O projeto exige a versão 20.9 ou maior; use a 22.
   - Conferir: `node -v` deve mostrar `v22.x`.
3. **pnpm 10**: no terminal, rode `npm install -g pnpm@10`. Feche e reabra o terminal.
   - Conferir: `pnpm -v` deve mostrar `10.x`. Se continuar "command not found", use `npx pnpm` no lugar de `pnpm`.
4. **Virtualização e WSL 2** (só Windows):
   - No Gerenciador de Tarefas → Desempenho → CPU, a linha **Virtualização** precisa estar **Habilitado**. Se estiver desabilitada, ative Intel VT-x ou AMD-V na BIOS (em máquina da empresa, peça à TI).
   - No PowerShell como administrador, rode `wsl --install` (ou `wsl --update`, se já tiver) e reinicie o computador.
5. **Docker Desktop**: baixe em https://www.docker.com/products/docker-desktop e instale com "Use WSL 2" marcado. Abra o programa e espere **Engine running** no canto inferior esquerdo. Não é preciso fazer login no Docker.
   - Em **Settings → Resources**, reserve pelo menos **6 GB de memória**: o Supabase local e o worker juntos usam de 3 a 5 GB.
   - Conferir: `docker run --rm hello-world` deve imprimir "Hello from Docker!".
6. **Stripe CLI** (opcional, só para testar planos e pagamentos): siga https://docs.stripe.com/stripe-cli. No Windows, a forma mais simples é baixar o `.zip`, extrair o `stripe.exe` numa pasta e adicioná-la ao PATH.
   - Conferir: `stripe --version`
7. **Aplicativo autenticador no celular** (Google Authenticator, Microsoft Authenticator ou 1Password): necessário para a conta master do painel, que exige verificação em duas etapas.

O **Supabase CLI** não precisa ser instalado à parte: ele vem como dependência do projeto e roda com `pnpm supabase …`.

Passo 2: criar as contas

Só a Anthropic é obrigatória. As demais você cria quando for testar a funcionalidade correspondente.

Conta	Para quê	Onde	O que guardar	Obrigatória?
Anthropic	Análise dos contratos pela IA	https://platform.claude.com	Chave `sk-ant-…` (API Keys) e créditos em Billing	Sim
Stripe (modo de teste)	Planos, checkout, portal	https://dashboard.stripe.com	Chave `sk_test_…` (Developers → API keys)	Só para planos
Brevo	Receber de verdade os avisos no Gmail	https://www.brevo.com	Login SMTP, chave SMTP e remetente verificado	Não
Sentry	Ver erros do app e do worker	https://sentry.io	DSN de um projeto Next.js e de um Node	Não
Supabase (nuvem)	Só se não puder usar Docker para o banco	https://supabase.com	Veja o Passo 7	Não

Anthropic, em detalhe:

1. Crie a conta em https://platform.claude.com.
2. Em **Billing**, adicione créditos. A API é cobrada à parte da assinatura do Claude.ai; um valor pequeno cobre muitos testes, porque cada contrato de teste custa alguns centavos de dólar.
3. Em **API Keys**, crie uma chave e copie o valor (`sk-ant-…`). Ela só aparece uma vez.

Nunca compartilhe a chave da Anthropic, a secret key do Supabase ou a chave do Stripe em conversas, prints ou repositórios. A publishable key do Supabase e o DSN do Sentry são públicos por natureza.

Passo 3: obter o código e instalar as dependências

1. Descompacte o zip mais recente (`etapa13-seguranca.zip`) numa pasta **sem espaços nem acentos no caminho**, por exemplo `C:\projetos\` . Cada zip traz o projeto completo; não misture pastas de etapas diferentes.
2. Descompacte também o `documentos-de-teste.zip` numa pasta à parte: ele traz os contratos usados no roteiro de testes.
3. Abra o Git Bash na pasta `app` e confira a estrutura:

```
cd /c/projetos/app
ls
# apps  evals  exemplos  packages  supabase  package.json  pnpm-
workspace.yaml  README.md
```

Pasta	Conteúdo
`apps/web`	App Next.js (telas, rotas, ações do servidor)
`apps/worker`	Worker de processamento e o `Dockerfile`
`packages/shared`	Código comum: erros, prompt, schema da análise, comparação, e-mail, calendário
`supabase`	Migrations, testes pgTAP, templates de e-mail, `config.toml` e `queries/custos.sql`
`evals`	Gabarito para medir a qualidade da análise
`exemplos`	Modelo de minuta de exemplo

4. Instale as dependências (sempre na raiz, a pasta `app`):

```
pnpm install
```

O comando instala o app, o worker e o pacote compartilhado de uma vez e copia o arquivo do visualizador de PDF para `apps/web/public/` . Se aparecer um aviso de atualização do pnpm, ignore: o projeto usa a versão 10.

5. (Recomendado) Crie um repositório Git local para acompanhar suas mudanças: `git init && git add . && git commit -m "Projeto inicial"` . O `.gitignore` já impede que arquivos `.env` sejam versionados.

Passo 4: subir o Supabase local

Com o Docker Desktop aberto (Engine running), rode na pasta `app` :

```
pnpm db:start
```

Na primeira vez, o comando baixa as imagens do Supabase (alguns GB) e leva de 5 a 15 minutos. Ele cria o banco e aplica todas as migrations da pasta `supabase/migrations` (tabelas, regras de segurança, fila, bucket de arquivos e planos). Ao final, mostra um quadro com endereços e chaves. **Copie para um bloco de notas:**

Item no quadro	Para que serve	Onde vai
API URL (`http://127.0.0.1:54321`)	Endereço do Supabase	Web e worker
Publishable key (`sb_publishable_…`)	Chave pública do app	Web
Secret key (`sb_secret_…`)	Chave do servidor e do worker	Web e worker
Studio URL (`http://127.0.0.1:54323`)	Painel do banco (tabelas, SQL)	Navegador
Mailpit URL (`http://127.0.0.1:54324`)	Caixa que recebe todos os e-mails de login	Navegador

Em versões mais antigas do CLI, as chaves aparecem como "anon key" e "service_role key"; elas funcionam do mesmo jeito. Para ver o quadro de novo a qualquer momento: `pnpm supabase status`.

Confira que deu certo (no Studio, http://127.0.0.1:54323):

1. **Table Editor**: existem as tabelas `offices`, `documents`, `risks`, `plans` e outras.
2. **Storage**: existe o bucket `documents`, privado.
3. **SQL Editor**: rode `select * from pgmq.list_queues();` e veja a fila `document_processing`.

O arquivo `supabase/config.toml` já deixa prontos, no ambiente local: a URL do app (`http://localhost:3000`), a confirmação de e-mail, os templates de e-mail em português e a verificação em duas etapas por aplicativo. Nada disso precisa ser configurado no painel.

Comandos do dia a dia:

- `pnpm supabase stop`: desliga o Supabase, mantendo os dados.
- `pnpm db:start`: liga de novo (rápido depois da primeira vez).
- `pnpm supabase migration up`: aplica migrations novas sem apagar dados.

- `pnpm db:reset`: **apaga tudo** e recria o banco do zero. Útil para recomeçar os testes; depois dele, é preciso cadastrar as contas e o administrador de novo.

Passo 5: configurar as variáveis de ambiente

São dois arquivos, criados a partir dos exemplos que vêm no projeto. Nenhum dos dois vai para o Git.

```
cp apps/web/.env.example apps/web/.env.local
cp apps/worker/.env.example apps/worker/.env
```

App web: `apps/web/.env.local`

Para o primeiro teste, bastam as quatro primeiras linhas; as demais podem ficar como estão até você chegar ao Passo 7.

```
# Supabase local (valores do quadro do pnpm db:start)
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
SUPABASE_SECRET_KEY=sb_secret_...
NEXT_PUBLIC_SITE_URL=http://localhost:3000

# Stripe (Passo 7; deixe como está se não for testar planos)
STRIPE_SECRET_KEY=sk_test_cole-aqui
STRIPE_WEBHOOK_SECRET=whsec_cole-aqui

# E-mails de aviso: console = só simula (salva em apps/web/.emails)
EMAIL_MODE=console
EMAIL_FROM=App <nao-responda@localhost>
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
RESEND_API_KEY=

# Sentry (opcional)
NEXT_PUBLIC_SENTRY_DSN=
```

Worker: `apps/worker/.env`

O worker roda dentro do Docker. Para ele, o seu computador se chama **`host.docker.internal`**, e não `127.0.0.1`: é o erro mais comum nesta etapa.

```
# Supabase local, visto de dentro do Docker
SUPABASE_URL=http://host.docker.internal:54321
SUPABASE_SECRET_KEY=sb_secret_...

# Anthropic (obrigatória para a análise)
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-5-5
ANTHROPIC_EFFORT=medium

WORKER_CONCURRENCY=2

# Links dos e-mails de aviso
SITE_URL=http://localhost:3000

# E-mails de aviso (console = só no terminal do worker)
EMAIL_MODE=console
EMAIL_FROM=App <nao-responda@localhost>
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
RESEND_API_KEY=

# Sentry (opcional)
SENTRY_DSN=
```

Referência de todas as variáveis

Variável	Arquivo	De onde vem	Obrigatória?
`NEXT_PUBLIC_SUPABASE_URL`	web	Quadro do `db:start` (API URL)	Sim
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	web	Quadro do `db:start` (Publishable key)	Sim
`SUPABASE_SECRET_KEY`	web e worker	Quadro do `db:start` (Secret key)	Sim
`NEXT_PUBLIC_SITE_URL`	web	`http://localhost:3000`	Sim
`SUPABASE_URL`	worker	`http://host.docker.internal:54321`	Sim
`ANTHROPIC_API_KEY`	worker	platform.claude.com → API Keys	Sim
`ANTHROPIC_MODEL`	worker	Padrão `claude-sonnet-5-5`	Não
`ANTHROPIC_EFFORT`	worker	`low`, `medium` (padrão) ou `high`	Não
`WORKER_CONCURRENCY`	worker	Documentos em paralelo (padrão 2)	Não
`SITE_URL`	worker	`http://localhost:3000`	Sim
`STRIPE_SECRET_KEY`	web	Stripe → Developers → API keys (`sk_test_…`)	Só para planos
`STRIPE_WEBHOOK_SECRET`	web	Saída do `stripe listen` (`whsec_…`)	Só para planos

Variável	Arquivo	De onde vem	Obrigatória?
`EMAIL_MODE`	web e worker	`console`, `smtp` ou `resend`	Não
`EMAIL_FROM`	web e worker	Remetente dos avisos	Não
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`	web e worker	Brevo → SMTP & API	Só com `smtp`
`RESEND_API_KEY`	web e worker	Resend (exige domínio)	Só com `resend`
`NEXT_PUBLIC_SENTRY_DSN` / `SENTRY_DSN`	web / worker	Sentry → projeto → Client Keys	Não

O app web só lê o `.env.local` quando o `pnpm dev` inicia, e o worker só lê o `.env` quando o contêiner sobe. **Depois de mudar qualquer valor, reinicie o processo correspondente.** Confira também erros de digitação: um `hhttps://` no lugar de `https://` já derrubou o app nos nossos testes.

Onde pegar cada valor, ferramenta por ferramenta

Supabase local (obrigatório)

1. Com o Supabase local ligado, rode na pasta `app`:

```
pnpm supabase status
```

2. Copie do quadro:

Linha do quadro	Variável	Arquivo
`API URL` (`http://127.0.0.1:54321`)	`NEXT_PUBLIC_SUPABASE_URL`	`apps/web/.env.local`
`API URL`, trocando `127.0.0.1` por `host.docker.internal`	`SUPABASE_URL`	`apps/worker/.env`
`Publishable key` (versões antigas: `anon key`)	`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	`apps/web/.env.local`
`Secret key` (versões antigas: `service_role key`)	`SUPABASE_SECRET_KEY`	**os dois** arquivos

Copie a chave inteira, sem espaços no começo ou no fim. As chaves locais são as mesmas sempre que você usa o mesmo computador; só mudam depois de reinstalar o Supabase local.

Supabase na nuvem (só se usar o banco na nuvem)

1. Em https://supabase.com/dashboard, abra o projeto.

2. **URL**: botão **Connect** no topo, ou **Project Settings → Data API → Project URL** (`https://<ref>.supabase.co`). Vai em `NEXT_PUBLIC_SUPABASE_URL` e, no worker, em `SUPABASE_URL` (a mesma URL, sem `host.docker.internal`).
3. **Chaves**: **Project Settings → API Keys**.
   - **Publishable key** (`sb_publishable_…`): clique em copiar → `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`.
   - **Secret keys**: clique em **Create new secret key** (ou no olho para revelar uma existente) e copie (`sb_secret_…`) → `SUPABASE_SECRET_KEY`, nos dois arquivos.
4. **Project ref** (para o `pnpm supabase link`): as 20 letras da URL do painel, depois de `/project/`.

Anthropic (obrigatória)

1. Em https://platform.claude.com, entre na conta.
2. Menu **API Keys → Create Key**. Dê um nome, como `local`, e clique em criar.
3. Copie o valor (`sk-ant-…`) **na hora**: ele não aparece de novo. Cole em `ANTHROPIC_API_KEY`, em `apps/worker/.env`.
4. Em **Billing**, confira que há créditos; sem eles, toda análise falha.

Stripe (só para testar planos)

1. Em https://dashboard.stripe.com, ligue o **modo de teste** (chave no canto superior direito).
2. **Developers → API keys → Secret key → Reveal test key**. Copie (`sk_test_…`) → `STRIPE_SECRET_KEY`, em `apps/web/.env.local`.
3. O `STRIPE_WEBHOOK_SECRET` não fica no painel no ambiente local: ao rodar `stripe listen …`, a primeira linha mostra `Your webhook signing secret is whsec_…`. Copie esse valor. Ele muda se você fizer login de novo no Stripe CLI.

Brevo (só para receber avisos de verdade)

1. Em https://app.brevo.com, menu da conta → **Senders, Domains & Dedicated IPs → Senders → Add a sender**. Confirme pelo e-mail que chega. Esse endereço vai em `EMAIL_FROM`, por exemplo `App <seu-email@gmail.com>`.
2. Menu da conta → **SMTP & API → aba SMTP**:
   - **Login** (algo como `9a1b2c@smtp-brevo.com`) → `SMTP_USER`. **Não é o seu e-mail.**
   - **Generate a new SMTP key**, dê um nome e copie → `SMTP_PASS`. Não confunda com a "API key", da outra aba.
   - `SMTP_HOST=smtp-relay.brevo.com` e `SMTP_PORT=587` são fixos.
3. Os mesmos valores vão nos dois arquivos, com `EMAIL_MODE=smtp`.

Sentry (opcional)

1. Em https://sentry.io, **Projects → Create Project**, plataforma **Next.js**, nome `app-web`. Na tela seguinte, copie o **DSN** (`https://…@….ingest.sentry.io/…`) → `NEXT_PUBLIC_SENTRY_DSN`.
2. Crie outro projeto, plataforma **Node.js**, nome `app-worker`, e copie o DSN → `SENTRY_DSN`, em `apps/worker/.env`.
3. Para achar um DSN depois: **Settings → Projects → (projeto) → Client Keys (DSN)**.

Valores fixos (não vêm de ferramenta nenhuma)

Variável	Valor local
`NEXT_PUBLIC_SITE_URL` (web) e `SITE_URL` (worker)	`http://localhost:3000`
`ANTHROPIC_MODEL`	`claude-sonnet-5-5`
`ANTHROPIC_EFFORT`	`medium` (ou `low` para testes mais baratos)
`WORKER_CONCURRENCY`	`2`
`EMAIL_MODE`	`console` (padrão) ou `smtp`

Conferir se ficou certo: abra os dois arquivos e veja se nenhuma linha obrigatória ainda tem `...`, `cole-aqui` ou está vazia. Ao subir, o worker recusa iniciar se faltar algo obrigatório e diz qual variável é.

Passo 6: subir o worker e o app

Você vai precisar de **dois terminais abertos ao mesmo tempo**, os dois na pasta `app`: o worker e o app ocupam o terminal enquanto rodam.

Terminal 1: worker

```
pnpm worker:build
pnpm worker:start
```

O `worker:build` monta a imagem Docker com OCR em português, LibreOffice, qpdf e poppler. Na primeira vez, leva de 3 a 10 minutos; depois, só é preciso repetir quando o código do worker ou do pacote compartilhado mudar (por exemplo, ao trocar para um zip novo). O `worker:start` lê o `apps/worker/.env`. Se ele subiu bem, a primeira linha do log é:

```
{"level":"info","msg":"worker iniciado","model":"claude-sonnet-5-5","effort":"medium","concurrency":2,"email":"console","sentry":false}
```

Os logs são em JSON e nunca mostram o conteúdo dos documentos: só ids, códigos, tempos e custos. Para parar, `Ctrl+C`; ele termina os documentos em andamento antes de sair.

Terminal 2: app web

```
pnpm dev
```

Espere "Ready" e abra http://localhost:3000. Sem login, você cai na tela **Entrar**.

Checagem rápida de que as três partes conversam:

Sinal	O que significa
A tela Entrar abre sem erro	App e Supabase conectados
O worker não mostra `falha ao ler a fila`	Worker e Supabase conectados
Um documento sai de "Na fila" em segundos (Passo 9)	Fila, worker e banco funcionando
Aparece "Analisando" e depois "Pronto"	Chave da Anthropic válida e com créditos

Para encerrar o dia: `Ctrl+C` nos dois terminais e `pnpm supabase stop`. No dia seguinte: `pnpm db:start`, `pnpm worker:start` e `pnpm dev`.

Passo 7: integrações opcionais

Faça cada uma só quando for testar a funcionalidade correspondente.

Stripe (planos e pagamentos, em modo de teste)

1. No painel do Stripe, ligue **Modo de teste** e copie a chave `sk_test_…` para `STRIPE_SECRET_KEY` em `apps/web/.env.local`.
2. Crie os produtos e preços dos planos (Essencial, Profissional e Escritório). O script lê a chave da linha de comando, não do `.env.local`:

```
STRIPE_SECRET_KEY=sk_test_... pnpm stripe:setup
```

3. Ative o **portal do cliente** em https://dashboard.stripe.com/test/settings/billing/portal: permita troca entre os três produtos, atualização da forma de pagamento e cancelamento.
4. Num **terceiro terminal**, repasse os eventos do Stripe para o app:

```
stripe login
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

5. Copie o `whsec_…` que o `stripe listen` mostra para `STRIPE_WEBHOOK_SECRET` e reinicie o `pnpm dev`. Deixe o `stripe listen` aberto enquanto testa.

Cartão de teste aprovado: `4242 4242 4242 4242`, qualquer data futura e qualquer CVC. Cartão recusado: `4000 0000 0000 0341`.

E-mail real dos avisos (Brevo)

No modo padrão (`console`), os avisos do app ficam em `apps/web/.emails/` como HTML e os do worker aparecem no terminal dele. Para recebê-los de verdade no Gmail:

1. No Brevo, verifique um remetente em **Senders & IP** e gere uma chave em **SMTP & API → SMTP**.
2. Nos **dois** arquivos (`apps/web/.env.local` e `apps/worker/.env`): `EMAIL_MODE=smtp`, `EMAIL_FROM` com o remetente verificado, `SMTP_USER` com o login do Brevo e `SMTP_PASS` com a chave.
3. Reinicie o `pnpm dev` e o worker.

Os e-mails de login do Supabase local (confirmação, senha, troca de e-mail) continuam indo para o Mailpit, o que é o esperado no ambiente local.

Sentry (monitoramento de erros)

Crie um projeto Next.js e um Node em https://sentry.io, copie o DSN de cada um para `NEXT_PUBLIC_SENTRY_DSN` (web) e `SENTRY_DSN` (worker) e reinicie os dois. A configuração não envia dados de usuário, cookies, cabeçalhos nem conteúdo das requisições.

Supabase na nuvem, no lugar do local

Use só se o Docker não puder rodar o banco. O worker continua precisando do Docker; se nem isso for possível, hospede o worker no Railway, Render ou Fly.io a partir do `apps/worker/Dockerfile`.

1. Crie o projeto em https://supabase.com na região **South America (São Paulo)**.

2. Na pasta `app`: `pnpm supabase login`, depois `pnpm supabase link --project-ref <ref>` (o ref são as 20 letras da URL do painel, não a URL inteira) e `pnpm supabase db push`.
3. Se o `db push` reclamar do `pgmq`, ative a extensão em **Database → Extensions** e rode de novo.
4. Em **Authentication → URL Configuration**: *Site URL* `http://localhost:3000` e *Redirect URLs* `http://localhost:3000/**`.
5. Em **Authentication → Emails → SMTP Settings**, configure um SMTP (Brevo ou Gmail com senha de app). Só assim os templates ficam editáveis: cole em **Templates** o conteúdo de `supabase/templates/confirmation.html`, `recovery.html` e `email_change.html`.
6. Nos arquivos `.env`, use a URL `https://<ref>.supabase.co` (também no worker, no lugar de `host.docker.internal`) e as chaves de **Project Settings → API Keys**.

Passo 8: primeiro acesso e contas de teste

No Supabase local, **qualquer e-mail funciona, mesmo inventado**: todos os e-mails de login caem no Mailpit (http://127.0.0.1:54324). Não é preciso usar aliases do Gmail. Crie estas quatro contas, cada uma com seu papel:

Conta	Papel	Como é criada
`ana@teste.local`	Titular do "Escritório A" (seu cliente de teste principal)	Cadastro normal
`carla@teste.local`	Membro do Escritório A	Convite da Ana
`bruno@teste.local`	Titular do "Escritório B" (para testar o isolamento)	Cadastro normal
`admin@teste.local`	Administrador da plataforma (painel `/admin`)	Cadastro normal + SQL

Cada navegador guarda uma sessão por site. Para ter várias contas abertas ao mesmo tempo, use perfis diferentes do Chrome ou uma janela normal e uma anônima.

Titular (Ana)

1. Em http://localhost:3000/cadastro, preencha nome, "Escritório A", `ana@teste.local` e uma senha com pelo menos 8 caracteres, incluindo letras maiúsculas, minúsculas e números (por exemplo, Teste1234). Marque o aceite dos termos e crie a conta.

2. No Mailpit, abra o e-mail "Confirme seu cadastro" e clique em **Confirmar e-mail**. Você entra em Documentos, com "Escritório A" no topo da barra lateral.
3. A avaliação gratuita começa sozinha: 14 dias e 5 documentos (visível em **Uso** e **Plano**).

Membro (Carla)

1. Logada como Ana, vá em **Configurações → Membros**, informe `carla@teste.local`, papel Membro, e clique em **Gerar convite**. Copie o link.
2. Numa janela anônima, abra o link, preencha nome e senha, aceite os termos e confirme pelo Mailpit.

Outro escritório (Bruno)

Repita o cadastro com `bruno@teste.local` e "Escritório B". Ele não pode ver nada do Escritório A.

Conta master (painel da plataforma)

1. Cadastre `admin@teste.local` (o cadastro cria um escritório; chame de "Plataforma (interno)" e não o use).
2. Logado com ela, vá em **Configurações → Minha conta → Verificação em duas etapas**, leia o QR code com o aplicativo autenticador e digite o código.
3. No Studio (http://127.0.0.1:54323), **SQL Editor**, rode:

```
insert into public.platform_admins (user_id)
select id from auth.users where email = 'admin@teste.local';
```

4. Saia, entre de novo (senha e código) e acesse http://localhost:3000/admin. Para qualquer outra conta, esse endereço responde "página não encontrada".

Depois de um `pnpm db:reset`, repita este passo inteiro: o banco volta vazio.

Passo 9: roteiro de testes por funcionalidade

Siga na ordem: cada bloco usa o que foi criado nos anteriores. Marque cada item ao concluir. Os arquivos citados estão no `documentos-de-teste.zip` e em `app/exemplos/`.

9.0 Antes de começar: libere documentos de teste

A avaliação gratuita permite só 5 documentos, e este roteiro usa mais que isso.

- [ ] Como `admin@teste.local`, em **/admin → Escritórios → Escritório A → Avaliação**, defina uma data daqui a 60 dias e **100 documentos**. Salve.

9.1 Autenticação, termos e isolamento

- [ ] Cadastro sem marcar o aceite dos termos mostra erro no campo.
- [ ] "Esqueci minha senha" envia o e-mail ao Mailpit, e o link abre a tela de nova senha.
- [ ] Logado como Bruno, nenhum documento, cliente ou membro do Escritório A aparece.
- [ ] Como Carla (membro), a seção de convites e o botão de remover não aparecem.
- [ ] No Studio, tabela `legal_acceptances`: duas linhas por conta, com versão, data, IP e navegador.

9.2 Envio de documentos e erros

- [ ] Envie `contrato-locacao.pdf`: "Enviando" com porcentagem, "Conferindo o arquivo" e "Na fila".
- [ ] `nao-aceito.doc` é recusado antes do envio.
- [ ] `contrato-54-paginas.pdf`: "mais de 50 páginas".
- [ ] `contrato-corrompido.pdf` e `arquivo-falso.pdf`: "Não foi possível abrir o arquivo".
- [ ] `contrato-com-senha.pdf`: aceito no envio e recusado pelo worker como "protegido por senha".
- [ ] Um PDF qualquer acima de 25 MB é recusado antes do envio.

9.3 Análise e tela de duas colunas

- [ ] O `contrato-locacao.pdf` passa por Preparando e Analisando e fica **Pronto** em 1 a 3 minutos. No log do worker: `chamada de IA` (tokens e custo) e `job concluído`.
- [ ] `contrato-escaneado.pdf` passa por OCR e chega ao mesmo tipo de resultado.
- [ ] `contrato-locacao.docx` aparece convertido em PDF na visualização.
- [ ] `contrato-restrito-sem-senha.pdf` é aceito (a proteção é removida).
- [ ] Aba **Riscos**: cerca de 12 riscos (veja o `LEIA-ME.txt`). Clicar num risco rola o PDF e destaca o trecho na cor do nível.
- [ ] Aba **Dados**: partes com CNPJ (inclusive o alfanumérico `12.ABC.345/01DE-35`), valor, vigência, multas e foro. Edite um campo e salve; "Restaurar dados da IA" desfaz.
- [ ] Aba **Resumo**: editar, salvar e "Copiar resumo".
- [ ] Na lista, a busca por "Aurora" e por `11.222.333/0001-81` encontra o documento.

9.4 Nosso cliente é…

- [ ] Escolha "Café Aurora… (Locatária)" e clique em **Refazer análise**. Os riscos passam a favorecer a locatária, e o resumo fala com ela ("você"). Os dados editados são mantidos, e **Uso** não aumenta.

9.5 Prazos e calendário

- [ ] Aba **Prazos**: fim da vigência (30/09/2029), aviso de não renovação (03/04/2029) e reajuste anual.
- [ ] Na aba Dados, mude o término para 30/09/2030: o aviso de não renovação vai para 03/04/2030.
- [ ] Adicione um prazo manual para daqui a 7 dias; o lembrete aparece no terminal do worker em até 5 minutos.
- [ ] Menu **Prazos → Baixar calendário (.ics)** e importe no Google Agenda.
- [ ] **Assinar na sua agenda → Gerar endereço**: abra o endereço no navegador (baixa o `.ics`). A assinatura real pelo Google só funciona com o app publicado, fora do `localhost`.

9.6 Clientes, responsáveis e revisão

- [ ] Na barra do documento, crie o cliente "Café Aurora", defina Carla como responsável e passe para "Em revisão".
- [ ] Marque a multa como "Em negociação" com uma nota; na lista, "Riscos altos em aberto" diminui.
- [ ] Menu **Clientes**: indicadores do cliente. Na página dele, envie um documento, que já sai vinculado.
- [ ] Menu **Acompanhamento** (Ana): carga por responsável; os números abrem a lista filtrada.

9.7 Versões e comparação

- [ ] No Word, edite o `contrato-locacao.docx`: multa de 12 para 3 aluguéis, apague a cláusula 11 e inclua uma cláusula de confidencialidade. Salve como v2.
- [ ] No documento original, **Versões → Enviar nova versão**, com a v2. Ela herda cliente, responsável e perspectiva.
- [ ] **Comparar com a v1**: preferência resolvida, confidencialidade nova, cláusula 9.1 alterada palavra a palavra, e a multa com a situação "Em negociação" herdada.
- [ ] Envie um contrato totalmente diferente como "nova versão": o worker recusa com "não parece ser uma nova versão", sem custo.
- [ ] Em **Uso**, as versões v2 a v4 não contam.

9.8 Playbook

- [ ] Em **Configurações → Playbook**, use o exemplo "Foro na nossa comarca" e salve.
- [ ] Reprocesse o contrato: o foro de Manaus aparece com a marca "Regra do escritório" e nível alto.

9.9 Minutas

- [ ] Em **Modelos**, envie `exemplos/modelo-notificacao-rescisao.docx`. O retorno mostra as variáveis automáticas e 2 manuais (`cidade` e `data_desocupacao`).
- [ ] Preencha telefone e OAB em **Minha conta** e endereço em **Escritório**.
- [ ] No contrato, **Gerar minuta**: o DOCX baixado traz partes, CNPJ, datas, lista de partes, seu nome e OAB; campos vazios saem como `__________`.

9.10 Qualidade

- [ ] Marque riscos como **Útil** e **Não procede** e preencha "Faltou apontar algum risco?".
- [ ] Em **/admin → Qualidade**, aparecem só as contagens por versão do prompt.

9.11 Planos e pagamento (exige o Stripe do Passo 7)

- [ ] Em **Plano**, os três planos mostram o preço vindo do Stripe.
- [ ] **Assinar** o Essencial com o cartão `4242…`: ao voltar, o plano aparece como atual, com 30 documentos.
- [ ] **Gerenciar assinatura** abre o portal; troque para o Profissional e confira o novo limite.
- [ ] No Essencial, o Playbook avisa que não entra nas análises; no Profissional, volta a valer.

9.12 Painel da plataforma

- [ ] **Novo cliente**: `cliente2@teste.local`, com 30 dias e 10 documentos. Abra o convite numa janela anônima: a pessoa entra como **titular**.
- [ ] Registre um **contrato por boleto** de 1 ano para esse escritório: no app do cliente, Plano mostra "Contrato até…" e some o botão de assinar.
- [ ] Um contrato que vence em 2 dias gera o aviso de vencimento (terminal do worker).
- [ ] **Suspender**: o cliente vê o aviso e não envia documentos, mas acessa os existentes. Reative.
- [ ] **Auditoria** lista todas as ações acima.

9.13 Conta, papéis e exclusões

- [ ] Minha conta: trocar a senha (pede a atual e encerra as outras sessões abertas), trocar o e-mail (link no Mailpit) e "Sair de todos os dispositivos".
- [ ] Ana promove Carla a administradora e depois transfere a titularidade.
- [ ] Carla exclui a própria conta com a senha; os documentos dela continuam no escritório.
- [ ] Exportar dados (titular): o JSON traz documentos, análises e registros de acesso.

- [ ] Excluir o escritório B: aviso em todas as páginas e envios bloqueados. Para testar a execução sem esperar 7 dias, rode no Studio `update offices set deletion_scheduled_for = now() where name = 'Escritório B';` e aguarde até 15 minutos com o worker ligado.

Passo 10: testes automatizados, qualidade e custo

Rode tudo na pasta `app`, com o Supabase local ligado.

Comando	O que testa	Resultado esperado
`pnpm typecheck`	Tipos do app, do worker e do pacote compartilhado	Nenhum erro
`pnpm db:test`	Migrations e regras de segurança do banco (pgTAP)	355 testes passando
`pnpm test`	Pacote compartilhado, worker e cobrança do app	Todos passando
`docker run --rm --user root app-worker node --experimental-strip-types --test apps/worker/src/pipeline/prepare.test.ts`	OCR, LibreOffice e qpdf reais, dentro da imagem	8 testes passando
`pnpm eval`	Qualidade da análise contra o gabarito (chama a API de verdade)	Relatório com as porcentagens

No `pnpm test`, os testes de OCR do worker são pulados fora da imagem Docker; o comando `docker run` acima roda esses testes lá dentro. Para repetir a validação de segurança, rode também pnpm security:audit (dependências e testes do banco); antes de qualquer pnpm supabase db push no banco da nuvem, faça um backup com pnpm db:backup.

Qualidade da IA (`pnpm eval`)

Usa a `ANTHROPIC_API_KEY` do `apps/worker/.env` e roda os casos de `evals/casos/` (o contrato de locação, em versão neutra e do ponto de vista da locatária). Cada caso custa uma análise. O terminal mostra: % de campos extraídos corretamente, % de riscos do gabarito encontrados, % com nível adequado, % de trechos localizados no PDF, custo e tempo, além da diferença em relação à última execução. A primeira execução vira a sua **linha de base**; rode de novo sempre que mudar o prompt, o modelo ou o esforço.

Custo por documento

No SQL Editor do Studio, abra e rode as consultas de `supabase/queries/custos.sql`: custo e tempo por documento, média e percentis dos últimos 30 dias, custo por página e custo por escritório e mês. É a base para definir os preços dos planos.

CI no GitHub

Ao subir o repositório para o GitHub, o `.github/workflows/ci.yml` roda tipos, testes, build, os testes do banco num Supabase local e os testes de OCR dentro da imagem, a cada envio de código. Não exige configuração além de ter o repositório lá.

Solução de problemas

A maioria destes erros já apareceu durante o desenvolvimento; a coluna da direita traz a solução que funcionou.

Sintoma	Causa provável	Solução
`pnpm: command not found`	pnpm não instalado ou terminal antigo	`npm install -g pnpm@10`, feche e reabra o terminal; ou use `npx pnpm`
`docker: command not found` ou "Virtualization support not detected"	Docker ausente ou virtualização desligada	Passo 1, itens 4 e 5; em máquina corporativa, a TI habilita na BIOS
Erro 500 em `dockerDesktopLinuxEngine`	Docker Desktop fechado ou iniciando	Abra o Docker Desktop e espere "Engine running"; se persistir, `wsl --shutdown` no PowerShell e reabra
`Invalid supabaseUrl` no `pnpm dev`	Erro de digitação na URL (ex.: `hhttps`)	Corrija o `.env.local` e reinicie o `pnpm dev`
Documento parado em "Na fila"	Worker desligado ou sem acesso ao Supabase	Suba o worker (Passo 6); confira `SUPABASE_URL=http://host.docker.internal:54321` e a secret key
Log do worker: `falha ao ler a fila`	URL, chave ou migrations	Confira o `.env` do worker e rode `pnpm supabase migration up`
`job falhou` com `401 invalid x-api-key`	Chave da Anthropic ausente ou errada	Corrija `ANTHROPIC_API_KEY` e reinicie o worker; depois, "Reprocessar"

Sintoma	Causa provável	Solução
Erro de IA indisponível ou `429`	Sem créditos ou limite de uso da API	Adicione créditos em platform.claude.com; o worker tenta de novo sozinho em 1, 2 e 4 minutos
Mudança no código não aparece no worker	Imagem antiga	`pnpm worker:build` e suba de novo
Visualizador de PDF não carrega	Falta `apps/web/public/pdf.worker.min.mjs`	Rode `pnpm install` de novo
Link do e-mail dá "link inválido"	Link expirado (1 h) ou já usado	Peça outro. No Supabase na nuvem com templates padrão, abra o link no mesmo navegador do pedido
Portas `54321` ou `3000` ocupadas	Outro Supabase ou app rodando	Pare o outro processo; `pnpm supabase stop` em outros projetos
Plano não muda depois do checkout	Webhook não chegou	Confira o `stripe listen` aberto, o `whsec_…` certo no `.env.local` e o `pnpm dev` reiniciado
Planos "Indisponível no momento"	Preços não criados no Stripe	`STRIPE_SECRET_KEY=sk_test_... pnpm stripe:setup`
"reprocessado 3 vezes nas últimas 24 horas"	Limite de custo por documento	Envie o arquivo de novo como documento novo, ou `pnpm db:reset` no ambiente de teste
"limite de envios com erro"	20 falhas em 24 h no escritório	Em /admin → escritório → Limite de falhas, aumente
"O escritório usou todos os documentos"	Cota da avaliação (5) esgotada	Em /admin → escritório → Avaliação, aumente os documentos
`/admin` responde "não encontrada"	Conta sem registro de administrador	Rode o SQL do Passo 8 com o e-mail certo
`/admin` pede verificação	Sessão sem a 2ª etapa	Saia e entre de novo com o código do autenticador
Assinatura do calendário não funciona no Google	App em `localhost`	Normal no ambiente local; use o download `.ics`
Avisos do worker não aparecem em `.emails`	No Docker, o modo `console` escreve dentro do contêiner (o erro permission denied, mkdir '/app/.emails' foi corrigido no pacote etapa13: rode pnpm worker:build)	Veja o terminal do worker ou use `EMAIL_MODE=smtp`

O que você pode ter esquecido

Itens que não estavam no pedido, mas fazem diferença.

- **Troque a secret key do projeto Supabase na nuvem.** Ela apareceu na nossa conversa. Em Project Settings → API Keys, crie uma nova e apague a antiga, ou apague o projeto se ele foi só de teste.
- **Use só contratos fictícios ou anonimizados nos testes.** O texto de cada documento é enviado à API da Anthropic. Se testar com um contrato real, exclua-o definitivamente depois, pela página do documento.
- **Economize nos testes:** `ANTHROPIC_EFFORT=low` reduz custo e tempo enquanto você testa telas e fluxos. Volte para `medium` para avaliar a qualidade e rodar o `pnpm eval`.

- **Relógio do computador certo.** O código do autenticador depende da hora exata; com o relógio atrasado, a verificação em duas etapas falha. O cálculo dos prazos também usa a data atual.
- **Rede corporativa.** Proxy ou antivírus da empresa podem bloquear o download das imagens Docker e dos pacotes do pnpm. Se o `db:start` ou o `pnpm install` travarem, teste em outra rede ou peça liberação à TI.
- **Ao trocar para um zip novo:** copie os dois arquivos `.env`, rode `pnpm install`, `pnpm supabase migration up` e `pnpm worker:build`, nesta ordem.
- **Dados locais são descartáveis.** `pnpm db:reset` apaga tudo. Se quiser guardar algo de um teste, use **Escritório → Zona de risco → Exportar dados** antes.
- **Textos jurídicos provisórios.** O cadastro exige o aceite da versão `2026-09-provisoria`. Ao publicar a redação final, atualize `apps/web/src/content/legal/` e crie uma migration com a nova versão (instruções no cabeçalho da migration de termos).
- **Chaves fora de prints e do Git.** Os arquivos `.env` e `.env.local` já estão no `.gitignore`; confira antes de subir o repositório.
- **Antes de produção** (fora deste guia): domínio próprio com Resend, hospedagem do app e do worker, `NEXT_PUBLIC_SITE_URL` e `SITE_URL` com o domínio público, Stripe em modo live, webhook de produção, preços finais e textos jurídicos revisados.

Todos os comandos explicados

Rode todos no **Git Bash**, dentro da pasta **`app`** (a que tem `apps`, `packages` e `supabase`). Para conferir onde você está: `pwd` deve terminar em `/app`. Os comandos que agem no banco da **nuvem** estão marcados; todos os outros afetam só o seu computador.

Conferir o Docker

Comando	O que faz	Quando usar	Saída esperada
`docker info`	Mostra se o Docker está rodando	Antes de `db:start` ou `worker:start`, se algo falhar	Várias linhas com `Server Version`; erro se o Docker Desktop estiver fechado
`docker run --rm hello-world`	Roda um contêiner de teste e o apaga	Só na instalação, para confirmar que o Docker funciona	`Hello from Docker!`

Instalar dependências

Comando	O que faz	Quando usar	Saída esperada
`pnpm install`	Baixa as bibliotecas do app, do worker e do pacote compartilhado e copia o leitor de PDF	Na primeira vez e sempre que trocar para um zip novo	`Done in … s`

Supabase local

Comando	O que faz	Quando usar	Saída esperada
`pnpm db:start`	Liga o banco, a autenticação, os arquivos, o Studio e o Mailpit no Docker; na primeira vez, aplica todas as migrations	No início de cada dia de trabalho	Quadro com `API URL`, chaves, `Studio URL` e `Mailpit URL`
`pnpm supabase status`	Mostra de novo o quadro de endereços e chaves	Quando precisar das chaves para os `.env`	O mesmo quadro
`pnpm supabase stop`	Desliga o Supabase local, mantendo os dados	No fim do dia	`Stopped supabase local development setup.`
`pnpm supabase migration up`	Aplica só as migrations novas, sem apagar dados	Depois de trocar para um zip novo	`Applying migration …` para cada nova; nada, se já estiver em dia
`pnpm db:reset`	**Apaga todos os dados locais** e recria o banco do zero com todas as migrations	Para recomeçar os testes do zero	`Finished supabase db reset`

Depois de um `db:reset`, cadastre de novo as contas e o administrador (Passo 8).

Testes

Comando	O que faz	Quando usar	Saída esperada
`pnpm db:test`	Roda os 355 testes do banco: regras de acesso, isolamento, ataques simulados	Depois de mudanças no banco e antes de qualquer deploy	`All tests successful` e `Result: PASS`
`pnpm --filter @app/shared test`	Testa só o pacote compartilhado (erros, CPF/CNPJ, prompt, comparação, calendário)	Para testar uma parte isolada	`# pass 43` e `# fail 0`
`pnpm test`	Testa o pacote compartilhado, o worker e o app de uma vez	Antes de enviar código	Todos com `# fail 0`
`pnpm typecheck`	Confere os tipos do TypeScript nas três partes	Antes de enviar código	Nenhuma linha de erro
`pnpm security:audit`	Procura vulnerabilidades nas bibliotecas e roda os testes do banco	Antes de qualquer deploy	`No known vulnerabilities found` e `Result: PASS`
`pnpm eval`	Mede a qualidade da análise contra o gabarito, **usando a API da Anthropic (tem custo)**	Ao mudar o prompt, o modelo ou o esforço	Porcentagens de extração e riscos, custo e tempo

Worker e app

Comando	O que faz	Quando usar	Saída esperada
`pnpm worker:build`	Monta a imagem Docker do worker com OCR, LibreOffice e qpdf	Na primeira vez e depois de trocar para um zip novo	`naming to docker.io/library/app-worker` no fim

Comando	O que faz	Quando usar	Saída esperada
`pnpm worker:start`	Liga o worker, que lê o `apps/worker/.env` e processa a fila	Sempre que for usar o app, num terminal só para ele	`"msg":"worker iniciado"`
`pnpm dev`	Liga o app em http://localhost:3000, recarregando a cada mudança no código	Sempre que for usar o app, em outro terminal	`Ready in …`

`worker:start` e `dev` ocupam o terminal enquanto rodam: para parar, `Ctrl+C`. Por isso, use dois terminais.

Supabase na nuvem

Comando	O que faz	Quando usar	Saída esperada
`pnpm supabase login`	Autoriza o Supabase CLI na sua conta (abre o navegador)	Uma vez por computador	`You are now logged in.`
`pnpm supabase link --project-ref vctzprdtimmbgllgytux`	Vincula esta pasta ao seu projeto na nuvem (troque pelo seu ref, se for outro)	Uma vez por pasta	`Finished supabase link.`
`pnpm db:backup`	**Nuvem:** salva a estrutura e os dados em `backups/`	Antes de cada `db push`	Arquivos `backups/schema.sql` e `backups/dados.sql`
`pnpm supabase db push`	**Nuvem:** aplica no banco da nuvem as migrations que faltam	Ao publicar uma versão nova	Lista das migrations, pergunta `[Y/n]`, depois `Finished supabase db push.`

Atenção: depois do `link`, o `db push` sempre age no banco da nuvem, inclusive no ambiente do advogado. Os comandos do Supabase local continuam agindo só no seu computador.

Stripe (só para testar planos)

Comando	O que faz	Quando usar	Saída esperada
`STRIPE_SECRET_KEY=sk_test_...` `pnpm stripe:setup`	Cria no Stripe de teste os produtos e preços dos três planos	Uma vez	Os três planos com os preços criados ou já existentes
`stripe login`	Autoriza o Stripe CLI na sua conta	Uma vez por computador	Mensagem de pareamento concluído
`stripe listen --forward-to localhost:3000/api/stripe/webhook`	Repassa os eventos do Stripe para o app local	Enquanto testa pagamentos, num terceiro terminal	`Your webhook signing secret is whsec_…`

Referência rápida

Primeira vez (na pasta `app`):

```
pnpm install
pnpm db:start
cp apps/web/.env.example apps/web/.env.local     # preencher (Passo 5)
cp apps/worker/.env.example apps/worker/.env     # preencher (Passo 5)
pnpm worker:build
pnpm worker:start                                # terminal 1
pnpm dev                                         # terminal 2
```

Dia a dia: `pnpm db:start`, `pnpm worker:start` (terminal 1) e `pnpm dev` (terminal 2). Para desligar: `Ctrl+C` nos dois e `pnpm supabase stop`.

Comando	Para quê
`pnpm supabase status`	Ver de novo URLs e chaves do Supabase local
`pnpm supabase migration up`	Aplicar migrations novas sem apagar dados
`pnpm db:reset`	Apagar tudo e recriar o banco
`pnpm worker:build`	Reconstruir a imagem do worker depois de mudanças no código
`STRIPE_SECRET_KEY=sk_test_... pnpm stripe:setup`	Criar produtos e preços no Stripe de teste
`stripe listen --forward-to localhost:3000/api/stripe/webhook`	Repassar eventos do Stripe (terminal 3)
`pnpm db:test` · `pnpm test` · `pnpm typecheck`	Testes automatizados
`pnpm eval`	Medir a qualidade da análise com a API real

Endereço	O que é
http://localhost:3000	App
http://localhost:3000/admin	Painel da plataforma (conta master com 2FA)
http://127.0.0.1:54323	Supabase Studio (tabelas e SQL)
http://127.0.0.1:54324	Mailpit (e-mails de login)
http://127.0.0.1:54321	API do Supabase (no worker: `host.docker.internal:54321`)